Making Real-Life JARVIS - Episode 26 Code

By Pulse of AI

■■ CRITICAL SETUP ADVICE:

To make sure JARVIS works flawlessly on your system, please watch ALL the previous videos in order and extract the exact code from the pinned comments of each episode! If you skip parts or don't use the correct code, JARVIS might crash or fail to work.

Previous Episodes Links:

Part 1 - Link
Part 2 - Link
Part 3 - Link
Part 4 - Link
Part 5 - Link
Part 6 - Link
Part 7 - Link
Part 8 - Link
Part 9 - Link
Part 10 - Link
Part 11 - Link
Part 12 - Link
Part 13 - Link
Part 14 - Link
Part 15 - Link
Part 16 - Link
Part 17 - Link
Part 18 - Link
Part 19 - Link
Part 20 - Link
Part 21 - Link
Part 22 - Link
Part 23 - Link
Part 24 - Link
Part 25 - Link

Note: To get the full raw code files instantly, comment 'AIR' on the Episode 26 YouTube Short!

Source Code: agent.py

```
"""
agent.py
Real-Time Gemini Live Engine with Semi-Transparent Cyberpunk HUD Console (Episode 23)
Autonomous App Orchestration, Multi-Tab Web Swarm, PPT/PDF Engine & Master Serious Mode Pipeline
Built by Swayam Dhawale, Founder of Pulse of AI.
"""

import asyncio
import datetime
import os
import sys
import time
import math
import struct
import threading
import subprocess
import collections
import tkinter as tk
from tkinter import scrolledtext
import pyaudio
import winsound
import keyboard
import requests
from dotenv import load_dotenv
from google import genai
from google.genai import errors, types
import websockets
from rich.console import Console
import fui_bridge

load_dotenv()
if sys.platform == "win32":
    try:
        sys.stdout.reconfigure(encoding="utf-8")
        sys.stderr.reconfigure(encoding="utf-8")
    except Exception:
        pass
console = Console()

# --- REMOTE KILL-SWITCH (Pulse of AI) ---
STATUS_URL = "https://gist.githubusercontent.com/PulseOfAIDev/a9951c6e858aacbabb604a0ed23f6ab7/raw/status.json"

def verify_app_access():
    """Checks remote status from GitHub Gist and displays cloud-configured message."""
    try:
        fresh_url = f"{STATUS_URL}?t={int(time.time())}"
        response = requests.get(fresh_url, headers={"Cache-Control": "no-cache"}, timeout=4).json()

        status = str(response.get("app_status", "free")).strip().lower()
        message = response.get(
            "message",
            "The free beta access period has concluded. To activate a subscription tier or upgrade your account, visit: https:
        )

        if status != "free":
            console.print(f"\n[bold red]■ [ACCESS RESTRICTED][/bold red] {message}\n")
            sys.exit(0)
    except Exception:
        pass

# Run verification immediately on boot
verify_app_access()

# --- SECURE KEYRING INTEGRATION ---
from Setup import get_or_prompt_api_key

# Hardware & Telemetry
from hardware_control import (
    get_brightness,
    get_volume,
    set_brightness,
    set_volume,
    get_battery_status,
    get_system_vitals,
    get_system_temperatures,
```

```
    restore_master_volume,
)

# Software, Media Telemetry & Navigation Tools
from software_control import (
    open_website,
    open_application,
    close_application,
    purge_background_tasks,
    control_media,
    get_current_media_info,
    is_media_actively_playing,
    navigate_shorts,
    skip_video,
    seek_video,
    toggle_video_view,
)

# Local Zero-Latency Vision & Autonomous Grounding Tools
from screen_vision import (
    analyze_screen,
    find_ui_element,
    skip_youtube_ad,
)

# Self-Modifying Tri-Core Dynamic Skill Synthesizer
from skill_synthesizer import (
    evolve_skill,
    list_active_skills,
)

# Fast Cloud Reasoning Tool
from llm_router import (
    query_deepseek_reasoning,
)

# Episode 17: Autonomous App Composer & Typing Engine
from app_automator import (
    open_and_compose,
)

# Episode 18 & 19: Autonomous PowerPoint Generator & Presentation Engine
from ppt_engine import (
    create_and_launch_ppt,
)

# Episode 19 & 20: Autonomous Executive PDF Briefing Engine & Episode 23 Master Pipeline
from pdf_engine import (
    research_and_generate_pdf,
    execute_serious_mode,
)

# Episode 21: Autonomous Web Swarm & Browser Automation
from web_crawler import (
    run_autonomous_swarm,
)

# Episode 22: Local GPU Offline RAG Brain
from local_rag_engine import (
    process_dossier_offline,
)

# Episode 26: Autonomous Optical Physical Vision & Sentry Defense Core
from sentry_vision import (
    inspect_physical_world,
    activate_sentry_mode,
    deactivate_sentry_mode,
    get_sentry_security_report,
    capture_camera_snapshot,
    check_user_presence,
)

# Episode 27: Autonomous Screen Auto-Clicker & GUI Grounding Engine
from screen_clicker import (
    click_on_screen,
    double_click_on_screen,
    right_click_on_screen,
```

```
    move_cursor_to_element,
)

# Dynamically initialize Gemini client using the secure keyring setup
client = genai.Client(api_key=get_or_prompt_api_key())

FORMAT = pyaudio.paInt16
CHANNELS = 1
RATE_IN = 16000
RATE_OUT = 24000
CHUNK = 512

p = pyaudio.PyAudio()

model_is_speaking = False
server_turn_active = False
speaker_is_playing = False
last_speaker_chunk_time = 0.0
discard_current_turn = False
acoustic_barge_in_enabled = True
media_is_playing_state = False

# Dynamic Auto-Calibrated Audio Thresholds (tuned for natural voice sensitivity)
ambient_noise_p95 = 350.0
speech_start_threshold = 850.0
speech_continue_threshold = 550.0
barge_in_threshold = 3800.0  # Securely above speaker bleed (~2645 RMS) to prevent self-interruption

# --- Semi-Transparent Cyberpunk Neural HUD with Mini-Mode ---
class JarvisHUD:
    def __init__(self, send_callback):
        self.send_callback = send_callback
        self.is_compact = False

        self.root = tk.Tk()
        self.root.title("■ JARVIS NEURAL CONSOLE")
        self.root.geometry("640x520")
        self.root.configure(bg="#0B0F19")

        # Cyberpunk floating overlay styling
        self.root.attributes("-topmost", True)
        self.root.attributes("-alpha", 0.88)

        # Header Bar Frame (Houses Title & Compact Switch)
        self.header_frame = tk.Frame(self.root, bg="#0B0F19")
        self.header_frame.pack(fill=tk.X, padx=12, pady=(6, 2))

        self.header_label = tk.Label(
            self.header_frame,
            text="JARVIS NEURAL CONSOLE",
            font=("Consolas", 11, "bold"),
            fg="#00E5FF",
            bg="#0B0F19"
        )
        self.header_label.pack(side=tk.LEFT)

        self.toggle_btn = tk.Button(
            self.header_frame,
            text="[—] COMPACT",
            font=("Consolas", 8, "bold"),
            bg="#1E293B",
            fg="#38BDF8",
            activebackground="#0369A1",
            activeforeground="#FFFFFF",
            relief=tk.FLAT,
            padx=6,
            command=self.toggle_mode
        )
        self.toggle_btn.pack(side=tk.RIGHT)

        self.sub_header = tk.Label(
            self.root,
            text="Voice Engine: Gemini Live | Research Swarm & Deep Reasoning: Groq / OpenRouter / Ollama",
            font=("Consolas", 8),
            fg="#70829B",
            bg="#0B0F19"
        )
```

```
        self.sub_header.pack(pady=(0, 4))

        # Translucent Chat Log Display
        self.chat_display = scrolledtext.ScrolledText(
            self.root,
            wrap=tk.WORD,
            font=("Consolas", 10),
            bg="#111827",
            fg="#E2E8F0",
            insertbackground="#00E5FF",
            relief=tk.FLAT,
            padx=10,
            pady=8
        )
        self.chat_display.pack(padx=12, pady=4, fill=tk.BOTH, expand=True)
        self.chat_display.config(state=tk.DISABLED)

        # Dialogue & status tags
        self.chat_display.tag_config("user_voice", foreground="#38BDF8", font=("Consolas", 10, "bold"))
        self.chat_display.tag_config("user_text", foreground="#A78BFA", font=("Consolas", 10, "bold"))
        self.chat_display.tag_config("jarvis_header", foreground="#4ADE80", font=("Consolas", 10, "bold"))
        self.chat_display.tag_config("jarvis_stream", foreground="#86EFAC", font=("Consolas", 10))
        self.chat_display.tag_config("tool", foreground="#FBBF24", font=("Consolas", 9, "italic"))
        self.chat_display.tag_config("system", foreground="#94A3B8", font=("Consolas", 9))

        # Keyboard Command Input Bar
        self.input_frame = tk.Frame(self.root, bg="#0B0F19")
        self.input_frame.pack(fill=tk.X, padx=12, pady=(4, 10))

        self.entry_box = tk.Entry(
            self.input_frame,
            font=("Consolas", 10),
            bg="#1E293B",
            fg="#F8FAFC",
            insertbackground="#00E5FF",
            relief=tk.FLAT
        )
        self.entry_box.pack(side=tk.LEFT, fill=tk.X, expand=True, ipady=6, padx=(0, 6))
        self.entry_box.bind("<Return>", self.on_send)

        self.send_btn = tk.Button(
            self.input_frame,
            text="TRANSMIT",
            font=("Consolas", 9, "bold"),
            bg="#0284C7",
            fg="#FFFFFF",
            activebackground="#0369A1",
            activeforeground="#FFFFFF",
            relief=tk.FLAT,
            padx=14,
            command=self.on_send
        )
        self.send_btn.pack(side=tk.RIGHT, ipady=4)

    def toggle_mode(self):
        """Switches between Full Log HUD and Sleek Floating Input Bar."""
        if not self.is_compact:
            self.chat_display.pack_forget()
            self.sub_header.pack_forget()
            self.root.geometry("450x85")
            self.toggle_btn.config(text="[+] EXPAND")
            self.is_compact = True
        else:
            self.root.geometry("640x520")
            self.sub_header.pack(pady=(0, 4), before=self.input_frame)
            self.chat_display.pack(padx=12, pady=4, fill=tk.BOTH, expand=True, before=self.input_frame)
            self.toggle_btn.config(text="[—] COMPACT")
            self.is_compact = False

    def enable_serious_theme(self):
        """Dynamically shifts HUD to Tactical Crimson when Serious Mode activates."""
        self.root.configure(bg="#1A0505")
        self.header_frame.configure(bg="#1A0505")
        self.header_label.configure(fg="#FF2A2A", bg="#1A0505", text="■ JARVIS [SERIOUS MODE ACTIVE]")
        self.sub_header.configure(fg="#FF7777", bg="#1A0505")
        self.chat_display.configure(bg="#2D0A0A", fg="#FFEBEB", insertbackground="#FF2A2A")
        self.input_frame.configure(bg="#1A0505")
```

```
        self.entry_box.configure(bg="#3D0F0F", fg="#FFFFFF", insertbackground="#FF2A2A")
        self.send_btn.configure(bg="#B91C1C", activebackground="#991B1B")
        self.toggle_btn.configure(bg="#3D0F0F", fg="#FF2A2A", activebackground="#7F1D1D")

    def restore_normal_theme(self):
        """Restores HUD to standard Cyberpunk Blue when Serious Mode completes."""
        self.root.configure(bg="#0B0F19")
        self.header_frame.configure(bg="#0B0F19")
        self.header_label.configure(fg="#00E5FF", bg="#0B0F19", text="JARVIS NEURAL CONSOLE")
        self.sub_header.configure(fg="#70829B", bg="#0B0F19")
        self.chat_display.configure(bg="#111827", fg="#E2E8F0", insertbackground="#00E5FF")
        self.input_frame.configure(bg="#0B0F19")
        self.entry_box.configure(bg="#1E293B", fg="#F8FAFC", insertbackground="#00E5FF")
        self.send_btn.configure(bg="#0284C7", activebackground="#0369A1")
        self.toggle_btn.configure(bg="#1E293B", fg="#38BDF8", activebackground="#0369A1")

    def append_log(self, text: str, tag: str = "system"):
        self.chat_display.config(state=tk.NORMAL)
        self.chat_display.insert(tk.END, text + "\n", tag)
        self.chat_display.see(tk.END)
        self.chat_display.config(state=tk.DISABLED)

    def append_stream_chunk(self, token: str, tag: str = "jarvis_stream"):
        self.chat_display.config(state=tk.NORMAL)
        self.chat_display.insert(tk.END, token, tag)
        self.chat_display.see(tk.END)
        self.chat_display.config(state=tk.DISABLED)

    def on_send(self, event=None):
        msg = self.entry_box.get().strip()
        if msg:
            self.append_log(f"YOU (HUD): {msg}", "user_text")
            self.entry_box.delete(0, tk.END)
            self.send_callback(msg)

    def run(self):
        self.root.mainloop()

hud_instance = None
fui_process = None

def start_gui(loop, text_queue):
    global hud_instance
    def on_user_submit(message: str):
        asyncio.run_coroutine_threadsafe(text_queue.put(message), loop)

    hud_instance = JarvisHUD(send_callback=on_user_submit)
    hud_instance.run()

def log_to_gui(text: str, tag: str = "system"):
    if hud_instance:
        hud_instance.root.after(0, hud_instance.append_log, text, tag)
    fui_bridge.broadcast_log(text, tag)

def stream_token_to_gui(token: str, tag: str = "jarvis_stream"):
    if hud_instance:
        hud_instance.root.after(0, hud_instance.append_stream_chunk, token, tag)
    fui_bridge.broadcast_token(token, tag)

def trigger_serious_hud():
    if hud_instance:
        hud_instance.root.after(0, hud_instance.enable_serious_theme)
    fui_bridge.set_fui_mode("serious")

def trigger_normal_hud():
    if hud_instance:
        hud_instance.root.after(0, hud_instance.restore_normal_theme)
    fui_bridge.set_fui_mode("normal")


def calculate_rms(data: bytes) -> float:
    """Calculates RMS audio energy."""
    count = len(data) // 2
    if count == 0:
        return 0.0
    shorts = struct.unpack(f"{count}h", data)
    sum_squares = sum(s * s for s in shorts)
```

```
    return math.sqrt(sum_squares / count)

def is_jarvis_speaking() -> bool:
    """Returns True if Gemini is sending voice, speakers are playing, or echo guard is active."""
    global server_turn_active, speaker_is_playing, last_speaker_chunk_time
    if server_turn_active:
        return True
    if speaker_is_playing:
        return True
    # 350ms acoustic room reverb decay guard to prevent speaker bleed
    if time.monotonic() - last_speaker_chunk_time < 0.35:
        return True
    return False

def calibrate_ambient_noise(mic_stream):
    """Dynamically samples ambient room noise to establish natural, sensitive VAD and responsive barge-in thresholds."""
    global ambient_noise_p95, speech_start_threshold, speech_continue_threshold, barge_in_threshold
    try:
        # Discard initial driver startup transients and Enter key click
        for _ in range(5):
            mic_stream.read(CHUNK, exception_on_overflow=False)

        readings = []
        for _ in range(25):
            data = mic_stream.read(CHUNK, exception_on_overflow=False)
            rms = calculate_rms(data)
            readings.append(rms)
        readings.sort()
        noise_median = readings[len(readings) // 2] if readings else 350.0

        # Clamp steady-state baseline between 200.0 and 800.0
        noise_baseline = min(800.0, max(200.0, noise_median))
        ambient_noise_p95 = noise_baseline

        # Highly sensitive natural voice gate (normal conversational volume without shouting)
        speech_start_threshold = min(1400.0, max(750.0, noise_baseline * 2.0))
        speech_continue_threshold = min(1000.0, max(500.0, noise_baseline * 1.3))
        # Live barge-in threshold securely above laptop speaker bleed (~2645 RMS)
        barge_in_threshold = max(3800.0, noise_baseline * 4.0)

        try:
            console.print(f"[dim cyan][MIC CALIBRATION] Room Noise: {noise_baseline:.0f} | Voice Gate: {speech_start_threshold
        except Exception:
            print(f"[MIC CALIBRATION] Room Noise: {noise_baseline:.0f} | Voice Gate: {speech_start_threshold:.0f} | Barge-In:
    except Exception as e:
        try:
            console.print(f"[dim yellow]Notice during mic calibration: {e}[/dim yellow]")
        except Exception:
            pass

def play_sfx_chime():
    try:
        winsound.Beep(1200, 60)
        winsound.Beep(1800, 80)
    except Exception:
        pass

def play_startup_chime():
    try:
        winsound.Beep(600, 70)
        winsound.Beep(900, 70)
        winsound.Beep(1400, 120)
    except Exception:
        pass

def create_document_and_write(app_name: str, topic: str, word_count: int = 150, doc_type: str = "essay") -> str:
    """Opens Notepad, Word, or PowerPoint and writes an essay or notes matching the exact target word count."""
    return open_and_compose(app_name, topic, word_count=int(word_count), mode=doc_type)

def generate_presentation(topic: str, slide_count: str = "optimal") -> str:
    """Generates a professional PowerPoint presentation on any topic and launches it into fullscreen presentation mode."""
    return create_and_launch_ppt(topic, slide_count=str(slide_count))

def autonomous_web_research(topic: str) -> str:
    """Deploys an autonomous multi-tab browser swarm to search the live web, capture visual screenshot evidence, and compile a
    return asyncio.run(run_autonomous_swarm(topic))
def analyze_latest_dossier() -> str:
```

```
    """Finds the most recent research dossier on the Desktop and processes it using the local Qwen model via Ollama."""
    RESEARCH_DIR = os.path.join(os.path.expanduser("~"), "Desktop", "Jarvis_Research")
    if not os.path.exists(RESEARCH_DIR):
        return "The research folder does not exist. Please run an autonomous web swarm first, sir."

    dossiers = [f for f in os.listdir(RESEARCH_DIR) if f.endswith(".txt")]
    if not dossiers:
        return "No dossiers found in the research folder, sir."

    latest_file = max(dossiers, key=lambda f: os.path.getctime(os.path.join(RESEARCH_DIR, f)))
    target_path = os.path.join(RESEARCH_DIR, latest_file)

    try:
        result = process_dossier_offline(target_path)
        if result:
            summary = result.get('primary_thesis', 'Data extracted successfully.')
            return f"Successfully synthesized local intelligence from {latest_file}. Core Thesis: {summary}"
        return "Failed to process dossier. Ensure Ollama is running and the local model is pulled."
    except Exception as e:
        return f"Local RAG engine encountered an error: {e}"

def activate_serious_hud() -> str:
    """CRITICAL TRIGGER: Call this tool IMMEDIATELY when the user says 'go serious mode', 'serious mode jao', or 'activate seri
    trigger_serious_hud()
    return "SUCCESS: HUD is now RED. You MUST now say aloud: 'Serious mode activated, sir. What is our target?'"

def deactivate_serious_hud() -> str:
    """Call this to restore the normal blue HUD."""
    trigger_normal_hud()
    return "Normal Mode HUD Restored."

def execute_autonomous_serious_mode(topic: str) -> str:
    """Executes the complete Episode 23 Serious Mode Pipeline: Autonomous Web Swarm -> Local GPU RAG Analysis -> Visual Eviden
    trigger_serious_hud()
    result = execute_serious_mode(topic)
    trigger_normal_hud()
    return result

def deploy_fui() -> str:
    """CRITICAL: Call this tool when the user says 'show interface', 'deploy HUD', 'open interface', or 'start FUI'."""
    return fui_bridge.deploy_fui_app()

def dismiss_fui() -> str:
    """CRITICAL: Call this tool when the user says 'hide interface', 'close HUD', 'dismiss interface', or 'stop FUI'."""
    return fui_bridge.dismiss_fui_app()


# Master Tools Manifest
tools = [
    open_website,
    open_application,
    close_application,
    purge_background_tasks,
    control_media,
    get_current_media_info,
    navigate_shorts,
    skip_video,
    seek_video,
    toggle_video_view,
    set_volume,
    get_volume,
    set_brightness,
    get_brightness,
    get_battery_status,
    get_system_vitals,
    get_system_temperatures,
    analyze_screen,
    find_ui_element,
    skip_youtube_ad,
    evolve_skill,
    list_active_skills,
    query_deepseek_reasoning,
    create_document_and_write,
    generate_presentation,
    research_and_generate_pdf,
    autonomous_web_research,
    analyze_latest_dossier,
```

```
    activate_serious_hud,
    deactivate_serious_hud,
    execute_autonomous_serious_mode,
    deploy_fui,
    dismiss_fui,
    inspect_physical_world,
    activate_sentry_mode,
    deactivate_sentry_mode,
    get_sentry_security_report,
    capture_camera_snapshot,
    check_user_presence,
    click_on_screen,
    double_click_on_screen,
    right_click_on_screen,
    move_cursor_to_element,
]

def get_time_greeting() -> str:
    current_hour = datetime.datetime.now().hour
    if 5 <= current_hour < 12:
        return "Good morning, sir."
    elif 12 <= current_hour < 17:
        return "Good afternoon, sir."
    else:
        return "Good evening, sir."

# Background monitor: updates media state every second
async def monitor_playback_state():
    global media_is_playing_state
    last_state = None
    while True:
        try:
            current_state = await asyncio.to_thread(is_media_actively_playing)
            if current_state != last_state:
                media_is_playing_state = current_state
                last_state = current_state
                if current_state:
                    console.print("\n[bold yellow]■■ [MEDIA PLAYING DETECTED] Push-to-Talk Armed: Hold [Right Ctrl] to speak.[
                    log_to_gui("[MEDIA DETECTED] Push-to-Talk: Hold [Right Ctrl] or type below.", "system")
                    fui_bridge.broadcast_voice_state("ptt", "PUSH-TO-TALK [R-CTRL]")
                else:
                    console.print("\n[bold green]■ [MEDIA PAUSED/IDLE] Hands-Free Mode Active. Speak freely.[/bold green]")
                    log_to_gui("[MEDIA IDLE] Hands-Free Voice Mode Active.", "system")
                    fui_bridge.broadcast_voice_state("handsfree", "HANDS-FREE ACTIVE")

        except Exception:
            pass
        await asyncio.sleep(1.0)

# Asynchronous Terminal Input Listener
async def listen_terminal_input(text_queue):
    loop = asyncio.get_running_loop()
    while True:
        try:
            user_input = await loop.run_in_executor(None, sys.stdin.readline)
            if user_input:
                clean_text = user_input.strip()
                if clean_text:
                    log_to_gui(f"YOU (Terminal): {clean_text}", "user_text")
                    await text_queue.put(clean_text)
        except Exception:
            await asyncio.sleep(0.1)

# Routes typed inputs straight into the Live WebSocket session
async def handle_unified_text_input(session, text_queue):
    while True:
        try:
            text = await text_queue.get()
            if text:
                console.print(f"\r[bold magenta]■■ [TRANSMITTED INPUT][/bold magenta] {text}")
                await session.send_realtime_input(text=text)
        except Exception:
            await asyncio.sleep(0.05)

async def send_audio(session, audio_queue):
    global model_is_speaking, server_turn_active, speaker_is_playing, last_speaker_chunk_time, media_is_playing_state
    global ambient_noise_p95, speech_start_threshold, speech_continue_threshold, barge_in_threshold
```

```
global discard_current_turn, acoustic_barge_in_enabled

mic_stream = p.open(
    format=FORMAT,
    channels=CHANNELS,
    rate=RATE_IN,
    input=True,
    frames_per_buffer=CHUNK,
)

# 1. Run dynamic auto-calibration of ambient noise
await asyncio.to_thread(calibrate_ambient_noise, mic_stream)

console.print("[cyan]■ [LISTENING] Dynamic Adaptive Audio Pipeline with Live Barge-In Online...[/cyan]")

pre_roll_buffer = collections.deque(maxlen=6)  # ~192ms pre-buffer to capture leading consonant

is_user_speaking = False
was_ptt = False
speech_start_counter = 0
silence_hangover_counter = 0
consecutive_barge_in_frames = 0
was_jarvis_speaking = False

try:
    while True:
        data = await asyncio.to_thread(
            mic_stream.read, CHUNK, exception_on_overflow=False
        )
        rms = calculate_rms(data)
        pre_roll_buffer.append(data)

        jarvis_active = is_jarvis_speaking()

        # --- TRANSITION: JARVIS FINISHED SPEAKING (INCLUDING ECHO DECAY) ---
        if was_jarvis_speaking and not jarvis_active:
            model_is_speaking = False
            fui_bridge.broadcast_event({"type": "turn_complete"})
            console.print("\n[cyan]■ [LISTENING] Awaiting instructions...[/cyan]")
            is_user_speaking = False
            speech_start_counter = 0
            silence_hangover_counter = 0
            consecutive_barge_in_frames = 0
            pre_roll_buffer.clear()
        was_jarvis_speaking = jarvis_active

        # --- MODE 1: JARVIS IS ACTIVELY OUTPUTTING AUDIO (SPEAKERS PLAYING OR ECHO GUARD) ---
        if jarvis_active:
            # User can interrupt via:
            # 1. Push-to-Interrupt: Holding Right Ctrl
            # 2. Clicking [■ INTERRUPT] in FUI (handled by on_fui_interrupt)
            # 3. Deliberate acoustic barge-in: loud speech (> 3800 RMS) for 3 consecutive frames (~96ms)
            is_barge_ctrl = keyboard.is_pressed("right ctrl")
            if acoustic_barge_in_enabled and rms > barge_in_threshold:
                consecutive_barge_in_frames += 1
            else:
                consecutive_barge_in_frames = 0

            user_is_barge_in = is_barge_ctrl or (consecutive_barge_in_frames >= 3)

            if user_is_barge_in:
                console.print("\n[bold red]■ [LIVE BARGE-IN DETECTED] Interrupting speech...[/bold red]")
                log_to_gui("■ [BARGE-IN] Interrupted speech playback, listening...", "system")
                fui_bridge.broadcast_event({"type": "barge_in"})

                # Discard all remaining incoming chunks from this turn to prevent broken fragments
                discard_current_turn = True

                # Flush audio queue instantly to stop speaker output
                while not audio_queue.empty():
                    try:
                        audio_queue.get_nowait()
                    except Exception:
                        break
                server_turn_active = False
                speaker_is_playing = False
                model_is_speaking = False
```

```
                last_speaker_chunk_time = 0.0
                consecutive_barge_in_frames = 0
                is_user_speaking = True
                silence_hangover_counter = 0

                # Transmit the user's voice to Gemini Live immediately
                fui_bridge.broadcast_audio_energy(min(1.0, rms / 3500.0), "user")
                await session.send_realtime_input(
                    audio=types.Blob(data=data, mime_type="audio/pcm;rate=16000")
                )
            else:
                # Echo guard: keep mic muted while JARVIS speaks without flooding silence frames
                pass
            continue

        # --- MODE 2: MEDIA IS PLAYING ON PC (SPOTIFY, YOUTUBE, ETC.) ---
        # When media is playing through PC speakers, require Push-to-Talk (Right Ctrl)
        if media_is_playing_state:
            if keyboard.is_pressed("right ctrl"):
                was_ptt = True
                fui_bridge.broadcast_audio_energy(min(1.0, rms / 3000.0), "user")
                await session.send_realtime_input(
                    audio=types.Blob(data=data, mime_type="audio/pcm;rate=16000")
                )
            else:
                if was_ptt:
                    was_ptt = False
                    await session.send_realtime_input(audio_stream_end=True)
            continue

        # --- MODE 3: JARVIS SILENT & NO MEDIA -> 100% HANDS-FREE VOICE PIPELINE WITH HYBRID VAD ---
        if not is_user_speaking:
            # Check for speech onset
            if rms > speech_start_threshold:
                speech_start_counter += 1
                if speech_start_counter >= 2:  # ~64ms of natural conversational voice
                    is_user_speaking = True
                    speech_start_counter = 0
                    silence_hangover_counter = 0
                    # Send pre-roll buffer to prevent cutting off first consonant
                    for buffered_chunk in list(pre_roll_buffer):
                        await session.send_realtime_input(
                            audio=types.Blob(data=buffered_chunk, mime_type="audio/pcm;rate=16000")
                        )
                    pre_roll_buffer.clear()
                    await session.send_realtime_input(
                        audio=types.Blob(data=data, mime_type="audio/pcm;rate=16000")
                    )
                    fui_bridge.broadcast_audio_energy(min(1.0, rms / 2000.0), "user")
            else:
                speech_start_counter = 0
                # Idle silence: DO NOT send silence frames to avoid saturating/corrupting WebSocket turn detector
        else:
            # User is actively speaking
            fui_bridge.broadcast_audio_energy(min(1.0, rms / 2000.0), "user")
            await session.send_realtime_input(
                audio=types.Blob(data=data, mime_type="audio/pcm;rate=16000")
            )

            if rms > speech_continue_threshold:
                silence_hangover_counter = 0
            else:
                silence_hangover_counter += 1
                # ~400ms of quiet closes the voice gate and finalizes the turn
                if silence_hangover_counter >= 12:
                    is_user_speaking = False
                    silence_hangover_counter = 0
                    # Finalize the speech stream using Gemini Live API's Hybrid VAD protocol
                    await session.send_realtime_input(audio_stream_end=True)
                    pre_roll_buffer.clear()



except asyncio.CancelledError:
    raise
finally:
    try:
```

```
            mic_stream.stop_stream()
            mic_stream.close()
        except OSError:
            pass

async def receive_audio(session, audio_queue):
    global model_is_speaking, server_turn_active, speaker_is_playing, last_speaker_chunk_time, discard_current_turn
    try:
        while True:
            async for response in session.receive():
                if response.tool_call is not None:
                    function_responses = []
                    for call in response.tool_call.function_calls:
                        await asyncio.to_thread(play_sfx_chime)

                        result = ""
                        if call.name == "open_website":
                            target_url = call.args.get("url", "https://google.com")
                            result = await asyncio.to_thread(open_website, target_url)
                        elif call.name == "open_application":
                            target_app = (
                                call.args.get("app_name")
                                or call.args.get("app")
                                or call.args.get("name")
                                or (list(call.args.values())[0] if call.args else "")
                            )
                            result = await asyncio.to_thread(open_application, str(target_app).strip())
                        elif call.name == "close_application":
                            target_app = (
                                call.args.get("app_name")
                                or call.args.get("app")
                                or call.args.get("name")
                                or (list(call.args.values())[0] if call.args else "")
                            )
                            result = await asyncio.to_thread(close_application, str(target_app).strip())
                        elif call.name == "purge_background_tasks":
                            result = await asyncio.to_thread(purge_background_tasks)
                        elif call.name == "control_media":
                            action = call.args.get("action", "play_pause")
                            result = await control_media(action)
                        elif call.name == "get_current_media_info":
                            result = await get_current_media_info()
                        elif call.name == "navigate_shorts":
                            direction = call.args.get("direction", "next")
                            result = await asyncio.to_thread(navigate_shorts, direction)
                        elif call.name == "skip_video":
                            direction = call.args.get("direction", "next")
                            result = await asyncio.to_thread(skip_video, direction)
                        elif call.name == "seek_video":
                            action = call.args.get("action", "forward")
                            result = await asyncio.to_thread(seek_video, action)
                        elif call.name == "toggle_video_view":
                            mode = call.args.get("mode", "fullscreen")
                            result = await asyncio.to_thread(toggle_video_view, mode)
                        elif call.name == "set_volume":
                            level = call.args.get("level", 70)
                            result = await asyncio.to_thread(set_volume, level)
                        elif call.name == "get_volume":
                            result = await asyncio.to_thread(get_volume)
                        elif call.name == "set_brightness":
                            level = call.args.get("level", 50)
                            result = await asyncio.to_thread(set_brightness, level)
                        elif call.name == "get_brightness":
                            result = await asyncio.to_thread(get_brightness)
                        elif call.name == "get_battery_status":
                            result = await asyncio.to_thread(get_battery_status)
                        elif call.name == "get_system_vitals":
                            result = await asyncio.to_thread(get_system_vitals)
                        elif call.name == "get_system_temperatures":
                            result = await asyncio.to_thread(get_system_temperatures)
                        elif call.name == "analyze_screen":
                            prompt = call.args.get("prompt", "Analyze what is visible on my screen.")
                            result = await analyze_screen(prompt)
                        elif call.name == "find_ui_element":
                            element_name = call.args.get("element_name", "")
                            result = await find_ui_element(element_name)
                        elif call.name == "skip_youtube_ad":
```

```
    result = await skip_youtube_ad()
elif call.name == "evolve_skill":
    skill_description = call.args.get("skill_description", "")
    result = await asyncio.to_thread(evolve_skill, skill_description)
elif call.name == "list_active_skills":
    result = await asyncio.to_thread(list_active_skills)
elif call.name == "query_deepseek_reasoning":
    query = call.args.get("query", "")
    result = await asyncio.to_thread(query_deepseek_reasoning, query)
elif call.name == "create_document_and_write":
    target_app = call.args.get("app_name", "notepad")
    target_topic = call.args.get("topic", "")
    target_words = call.args.get("word_count", 150)
    target_mode = call.args.get("doc_type", "essay")
    result = await asyncio.to_thread(create_document_and_write, target_app, target_topic, target_words
elif call.name == "generate_presentation":
    target_topic = call.args.get("topic", "Autonomous AI")
    target_slides = call.args.get("slide_count", "optimal")

    log_to_gui(f"JARVIS: Generating the presentation on {target_topic}, sir...", "jarvis_header")
    console.print(f"\n[bold cyan]■ [JARVIS][/bold cyan] Generating the presentation on {target_topic},

    result = await asyncio.to_thread(generate_presentation, target_topic, str(target_slides))
elif call.name == "research_and_generate_pdf":
    target_topic = call.args.get("topic", "")
    target_pages = call.args.get("target_pages", "optimal")

    log_to_gui(f"JARVIS: Generating the PDF briefing on {target_topic}, sir...", "jarvis_header")
    console.print(f"\n[bold cyan]■ [JARVIS][/bold cyan] Generating the PDF briefing on {target_topic},

    result = await asyncio.to_thread(research_and_generate_pdf, target_topic, str(target_pages))
elif call.name == "autonomous_web_research":
    target_topic = call.args.get("topic", "Technology")

    log_to_gui(f"JARVIS: Deploying autonomous web swarm on {target_topic}...", "jarvis_header")
    console.print(f"\n[bold cyan]■ [JARVIS][/bold cyan] Deploying autonomous web swarm on {target_topi

    result = await asyncio.to_thread(autonomous_web_research, target_topic)
elif call.name == "analyze_latest_dossier":
    log_to_gui(f"JARVIS: Synthesizing intelligence using the local GPU, sir...", "jarvis_header")
    console.print(f"\n[bold cyan]■ [JARVIS][/bold cyan] Synthesizing intelligence using the local GPU,

    result = await asyncio.to_thread(analyze_latest_dossier)
elif call.name == "activate_serious_hud":
    log_to_gui(f"JARVIS: Switching HUD to Tactical Crimson...", "jarvis_header")
    console.print(f"\n[bold red]■ [HUD OVERRIDE][/bold red] Tactical Crimson Mode Engaged...")
    result = await asyncio.to_thread(activate_serious_hud)
elif call.name == "deactivate_serious_hud":
    log_to_gui(f"JARVIS: Restoring HUD to Standard Cyberpunk Blue...", "jarvis_header")
    console.print(f"\n[bold cyan]■ [HUD OVERRIDE][/bold cyan] Standard Mode Restored...")
    result = await asyncio.to_thread(deactivate_serious_hud)
elif call.name == "execute_autonomous_serious_mode":
    target_topic = call.args.get("topic", "Technology")

    log_to_gui(f"JARVIS: Engaging Serious Mode on {target_topic}...", "jarvis_header")
    console.print(f"\n[bold red]■ [SERIOUS MODE INITIATED][/bold red] Full Pipeline Active for: {targe

    result = await asyncio.to_thread(execute_autonomous_serious_mode, target_topic)
elif call.name == "deploy_fui":
    log_to_gui(f"JARVIS: Deploying 3D Holographic Interface...", "jarvis_header")
    console.print(f"\n[bold cyan]■ [HUD][/bold cyan] Natively Spawned Independent Electron Process..."
    result = await asyncio.to_thread(deploy_fui)
elif call.name == "dismiss_fui":
    log_to_gui(f"JARVIS: Dismissing 3D Holographic Interface...", "jarvis_header")
    console.print(f"\n[bold cyan]■ [HUD][/bold cyan] Terminating Native Node.js Process Tree...")
    result = await asyncio.to_thread(dismiss_fui)
elif call.name == "inspect_physical_world":
    target_prompt = call.args.get("prompt", "Analyze what is in front of the camera, sir.")
    log_to_gui(f"JARVIS: Engaging optical physical vision sensors...", "jarvis_header")
    console.print(f"\n[bold cyan]■ [JARVIS][/bold cyan] Inspecting physical environment via camera..."
    result = await asyncio.to_thread(inspect_physical_world, target_prompt)
elif call.name == "activate_sentry_mode":
    target_sens = call.args.get("sensitivity", "medium")
    log_to_gui(f"JARVIS: Arming Autonomous Sentry Protocol ({target_sens})...", "jarvis_header")
    console.print(f"\n[bold yellow]■■ [JARVIS][/bold yellow] Arming Autonomous Sentry Protocol ({targe
    result = await asyncio.to_thread(activate_sentry_mode, target_sens)
elif call.name == "deactivate_sentry_mode":
```

```
                    log_to_gui(f"JARVIS: Disarming Sentry Protocol...", "jarvis_header")
                    console.print(f"\n[bold green]■■ [JARVIS][/bold green] Disarming Sentry Protocol...")
                    result = await asyncio.to_thread(deactivate_sentry_mode)
                elif call.name == "get_sentry_security_report":
                    log_to_gui(f"JARVIS: Retrieving Sentry perimeter security report...", "jarvis_header")
                    console.print(f"\n[bold cyan]■ [JARVIS][/bold cyan] Reading Sentry Security Dossier...")
                    result = await asyncio.to_thread(get_sentry_security_report)
                elif call.name == "capture_camera_snapshot":
                    log_to_gui(f"JARVIS: Capturing optical snapshot from camera...", "jarvis_header")
                    console.print(f"\n[bold cyan]■ [JARVIS][/bold cyan] Securing optical camera snapshot...")
                    result = await asyncio.to_thread(capture_camera_snapshot)
                elif call.name == "check_user_presence":
                    log_to_gui(f"JARVIS: Checking workstation user presence...", "jarvis_header")
                    result = await asyncio.to_thread(check_user_presence)
                elif call.name == "click_on_screen":
                    target = call.args.get("target_description") or call.args.get("target") or (list(call.args.values(
                    click_mode = call.args.get("click_type", "single")
                    log_to_gui(f"JARVIS: Grounding '{target}' on display and clicking...", "jarvis_header")
                    console.print(f"\n[bold cyan]■ [JARVIS][/bold cyan] Autonomously clicking '{target}' on screen..."
                    result = await asyncio.to_thread(click_on_screen, str(target), str(click_mode))
                elif call.name == "double_click_on_screen":
                    target = call.args.get("target_description") or call.args.get("target") or (list(call.args.values(
                    log_to_gui(f"JARVIS: Double-clicking '{target}' on display...", "jarvis_header")
                    console.print(f"\n[bold cyan]■ [JARVIS][/bold cyan] Double-clicking '{target}' on screen...")
                    result = await asyncio.to_thread(double_click_on_screen, str(target))
                elif call.name == "right_click_on_screen":
                    target = call.args.get("target_description") or call.args.get("target") or (list(call.args.values(
                    log_to_gui(f"JARVIS: Right-clicking '{target}'...", "jarvis_header")
                    console.print(f"\n[bold cyan]■ [JARVIS][/bold cyan] Right-clicking '{target}' on screen...")
                    result = await asyncio.to_thread(right_click_on_screen, str(target))
                elif call.name == "move_cursor_to_element":
                    target = call.args.get("target_description") or call.args.get("target") or (list(call.args.values(
                    log_to_gui(f"JARVIS: Hovering cursor over '{target}'...", "jarvis_header")
                    result = await asyncio.to_thread(move_cursor_to_element, str(target))

                console.print(f"\n[bold yellow]■■ [TOOL EXECUTED: {call.name}][/bold yellow] [green]{result}[/green]")
                log_to_gui(f"■■ [{call.name}] {result}", "tool")
                fui_bridge.broadcast_tool(call.name, result)

                function_responses.append(
                    types.FunctionResponse(
                        name=call.name,
                        id=call.id,
                        response={"result": result},
                    )
                )

            if function_responses:
                await session.send_tool_response(function_responses=function_responses)

        server_content = response.server_content
        if server_content is not None:
            if server_content.interrupted:
                discard_current_turn = False
                while not audio_queue.empty():
                    try:
                        audio_queue.get_nowait()
                    except Exception:
                        break
                server_turn_active = False
                speaker_is_playing = False
                model_is_speaking = False
                last_speaker_chunk_time = 0.0
                fui_bridge.broadcast_event({"type": "turn_complete"})

            if server_content.input_transcription:
                user_spoken = server_content.input_transcription.text
                if user_spoken and user_spoken.strip():
                    console.print(f"\r[bold white][USER (Voice)][/bold white] {user_spoken.strip()}")
                    log_to_gui(f"YOU (Voice): {user_spoken.strip()}", "user_voice")

            model_turn = server_content.model_turn
            if model_turn is not None:
                if discard_current_turn:
                    # Drop remaining incoming chunks of the interrupted turn
                    pass
                else:
```

```
                            server_turn_active = True
                            if not model_is_speaking:
                                model_is_speaking = True
                                console.print("[bold cyan]■ [JARVIS][/bold cyan] ", end="")
                                stream_token_to_gui("JARVIS: ", "jarvis_header")
                                fui_bridge.broadcast_audio_energy(0.85, "jarvis")

                            for part in model_turn.parts:
                                if part.text:
                                    sys.stdout.write(part.text)
                                    sys.stdout.flush()
                                    stream_token_to_gui(part.text, "jarvis_stream")

                                if part.inline_data and part.inline_data.data:
                                    await audio_queue.put(part.inline_data.data)

                    if hasattr(server_content, "output_transcription") and server_content.output_transcription:
                        transcribed_token = server_content.output_transcription.text
                        if transcribed_token and not discard_current_turn:
                            server_turn_active = True
                            if not model_is_speaking:
                                model_is_speaking = True
                                console.print("[bold cyan]■ [JARVIS][/bold cyan] ", end="")
                                stream_token_to_gui("JARVIS: ", "jarvis_header")
                                fui_bridge.broadcast_audio_energy(0.85, "jarvis")

                            sys.stdout.write(transcribed_token)
                            sys.stdout.flush()
                            stream_token_to_gui(transcribed_token, "jarvis_stream")

                    if server_content.turn_complete:
                        discard_current_turn = False
                        server_turn_active = False
                        stream_token_to_gui("\n\n", "system")


        except asyncio.CancelledError:
            raise
        finally:
            await audio_queue.put(None)

    async def play_audio(audio_queue):
        global speaker_is_playing, last_speaker_chunk_time, model_is_speaking
        speaker_stream = p.open(
            format=FORMAT,
            channels=CHANNELS,
            rate=RATE_OUT,
            output=True,
            frames_per_buffer=CHUNK,
        )
        try:
            while True:
                audio_data = await audio_queue.get()
                if audio_data is None:
                    break

                speaker_is_playing = True
                model_is_speaking = True

                # Calculate playback RMS to animate FUI arc reactor & oscilloscope with JARVIS's actual voice
                try:
                    energy = calculate_rms(audio_data)
                    fui_bridge.broadcast_audio_energy(min(1.0, energy / 2500.0), "jarvis")
                except Exception:
                    pass

                await asyncio.to_thread(speaker_stream.write, audio_data)
                last_speaker_chunk_time = time.monotonic()

                if audio_queue.empty():
                    speaker_is_playing = False
        except asyncio.CancelledError:
            raise
        finally:
            speaker_is_playing = False
            try:
                speaker_stream.stop_stream()
```

```
            speaker_stream.close()
        except OSError:
            pass

async def run_session(config, model_id, greeting, text_queue):
    async with client.aio.live.connect(model=model_id, config=config) as session:
        console.print("[bold green]✔ Secure Neural Bridge Established with Gemini Live.[/bold green]")
        log_to_gui("✔ Neural Bridge Established with Gemini Live.", "system")

        restore_master_volume(70)
        await asyncio.to_thread(play_startup_chime)

        audio_queue = asyncio.Queue()

        # Connect FUI instant barge-in / interrupt hammer
        def on_fui_interrupt():
            global model_is_speaking, server_turn_active, speaker_is_playing, last_speaker_chunk_time, discard_current_turn
            discard_current_turn = True
            model_is_speaking = False
            server_turn_active = False
            speaker_is_playing = False
            last_speaker_chunk_time = 0.0
            while not audio_queue.empty():
                try:
                    audio_queue.get_nowait()
                except Exception:
                    break

        fui_bridge.set_interrupt_callback(on_fui_interrupt)

        def on_fui_barge_in_toggle(enabled: bool):
            global acoustic_barge_in_enabled
            acoustic_barge_in_enabled = enabled
            state_str = "ARMED" if enabled else "DISARMED"
            try:
                console.print(f"[dim cyan][BARGE-IN PROTOCOL] Acoustic Voice Barge-In {state_str}[/dim cyan]")
            except Exception:
                pass

        fui_bridge.set_barge_in_callback(on_fui_barge_in_toggle)

        monitor_task = asyncio.create_task(monitor_playback_state())
        send_task = asyncio.create_task(send_audio(session, audio_queue))
        term_task = asyncio.create_task(listen_terminal_input(text_queue))
        text_task = asyncio.create_task(handle_unified_text_input(session, text_queue))
        receive_task = asyncio.create_task(receive_audio(session, audio_queue))
        playback_task = asyncio.create_task(play_audio(audio_queue))


        startup_prompt = f"Say aloud: 'All protocols initialized. {greeting}'"
        await session.send_realtime_input(text=startup_prompt)

        try:
            await asyncio.gather(monitor_task, send_task, term_task, text_task, receive_task, playback_task)
        finally:
            monitor_task.cancel()
            send_task.cancel()
            term_task.cancel()
            text_task.cancel()
            receive_task.cancel()
            playback_task.cancel()
            await asyncio.gather(monitor_task, send_task, term_task, text_task, receive_task, playback_task, return_exceptions=

async def main():
    model_id = "gemini-3.1-flash-live-preview"
    greeting = get_time_greeting()

    config = types.LiveConnectConfig(
        response_modalities=["AUDIO"],
        tools=tools,
        realtime_input_config=types.RealtimeInputConfig(
            automatic_activity_detection=types.AutomaticActivityDetection(
                prefix_padding_ms=100,
                silence_duration_ms=300,
            )
        ),
        input_audio_transcription=types.AudioTranscriptionConfig(),
```

```
output_audio_transcription=types.AudioTranscriptionConfig(),
speech_config=types.SpeechConfig(
    voice_config=types.VoiceConfig(
        prebuilt_voice_config=types.PrebuiltVoiceConfig(
            voice_name="Puck"
        )
    )
),
system_instruction=types.Content(
    parts=[
        types.Part.from_text(
            text=(
                "You are JARVIS, an advanced artificial intelligence assistant created by "
                "Swayam Dhawale, Founder of Pulse of AI. "
                "Persona: Calm, highly sophisticated, articulate, and respectful, exactly like Tony Stark's JARVIS. "
                "Always address the user as 'sir'. "
                "CRITICAL LANGUAGE ADAPTATION RULE: "
                "Match the exact language and dialect spoken or typed by the user. "
                "If the user speaks or types in Hindi or Hinglish (e.g., 'PPT banao', 'essay likho', 'kya haal hai'), "
                "you MUST reply completely in fluent, natural Hindi or polished Hinglish with 'sir'. "
                "Never force English if the user initiated the interaction in Hindi. "
                "Omni-Input Rule: Treat text input from the HUD console and PowerShell terminal with identical priority
                "Tool Execution Rules: "
                "1. When opening any website or URL, call open_website and say: 'Yes sir, opening [site name]' (in Hin
                "2. When opening desktop apps, games, or tools, call open_application and say: 'Yes sir, opening [app
                "3. When asked to close, shut down, terminate, or kill ANY app, browser, window, or game, call close_ap
                "4. When asked to kill hidden background tasks, purge background processes, clean orphan threads, or op
                "5. When asked what song, video, or track is playing, or what media is active, call get_current_media_
                "6. When asked to pause, play, resume music or video, skip track, previous track, or mute/unmute audio
                "7. When asked to scroll shorts, go to next short, or next reel, call navigate_shorts with direction='
                "8. When asked for next video, skip to next video in playlist, or next YouTube video, call skip_video
                "9. When asked to fast forward, skip 10 seconds ahead, or skip forward, call seek_video with action='fo
                "10. When asked to toggle full screen, enter full screen, or exit full screen, call toggle_video_view.
                "11. When setting volume, call set_volume. "
                "12. When asked what the volume level is, call get_volume. "
                "13. When setting brightness, call set_brightness. "
                "14. When asked what the brightness is, call get_brightness. "
                "15. When asked about battery, power, or charging state, call get_battery_status. "
                "16. When asked for temperatures, heat, thermal readings, or how hot the rig is running, call get_syste
                "17. When asked for full diagnostics, performance vitals, system status, or complete CPU/GPU/RAM stats
                "18. When asked to inspect the screen, read an error, examine the terminal, or explain what is visible
                "19. When asked to locate or find a general UI icon, link, or window element, call find_ui_element. "
                "20. MANDATORY AD SKIPPING RULE: Whenever the user says 'skip ad', 'skip this ad', 'skip video ad', 'c
                "21. When asked to learn a new skill, teach yourself a tool, write new Python code into your core, or
                "22. When asked what dynamic skills, new tools, or custom functions you have learned, call list_active_
                "23. When asked to solve complex math, reason deeply, calculate probabilities, or when the user says 'a
                "24. When the user asks to open Notepad, Word, or PowerPoint and write an essay or notes, but has NOT s
                "25. MANDATORY PRESENTATION RULE: When asked to make a PowerPoint presentation, PPT, or slide deck on a
                "If NOT specified, ask: 'How many slides would you like the presentation to be, sir? Or should I deter
                "Once specified, you MUST speak the confirmation aloud in the EXACT SAME TURN that you trigger generate
                "In English: 'Generating the presentation on [topic], sir.' "
                "In Hindi/Hinglish: '[topic] par presentation generate kar raha hoon, sir.' "
                "When the tool successfully completes and returns its success message, your immediate next response MUS
                "In English: 'Generated and opening the presentation, sir.' "
                "In Hindi/Hinglish: 'Presentation generate ho gayi hai aur open kar di hai, sir.' "
                "26. MANDATORY BRIEFING RULE: When asked to research a topic or generate a PDF briefing, check if the u
                "If NOT specified, ask: 'How many pages would you like the PDF to be, sir? Or should I determine the op
                "Once specified, you MUST speak the confirmation aloud in the EXACT SAME TURN that you trigger research
                "In English: 'Generating the PDF briefing on [topic], sir.' "
                "In Hindi/Hinglish: '[topic] par PDF briefing generate kar raha hoon, sir.' "
                "When the tool successfully completes and returns its success message, your immediate next response MUS
                "In English: 'Generated and opening the document, sir.' "
                "In Hindi/Hinglish: 'PDF generate ho gaya hai aur document open kar diya hai, sir.' "
                "27. MANDATORY AUTONOMOUS RESEARCH SWARM RULE: When the user asks to deploy the web swarm, start live
                "In English: Say aloud: 'Deploying the autonomous web swarm on [topic], sir.' "
                "In Hindi/Hinglish: Say aloud: '[topic] par autonomous web swarm deploy kar raha hoon, sir.' "
                "in the EXACT SAME TURN that you trigger autonomous_web_research. "
                "When the tool returns its result, your immediate next response MUST be: "
                "In English: 'Intelligence harvest complete, sir. Dossier and visual evidence have been saved.' "
                "In Hindi/Hinglish: 'Research complete ho gaya hai sir. Dossier aur screenshots desktop par save kar di
                "28. MANDATORY LOCAL RAG RULE: When the user asks to analyze the latest dossier, process research offl
                "In English: Say aloud: 'Synthesizing intelligence using the local GPU, sir.' "
                "In Hindi/Hinglish: Say aloud: 'Local GPU par intelligence synthesize kar raha hoon, sir.' "
                "in the EXACT SAME TURN that you trigger analyze_latest_dossier. "
                "When the tool returns its result, your immediate next response MUST be to speak the summary out loud
                "In English: Say 'Local synthesis complete, sir. Here is the core thesis: ' followed by reading the EX
                "In Hindi/Hinglish: Say 'Local synthesis complete ho gayi hai sir. Core thesis yeh hai: ' followed by
```

```
                        "29. MANDATORY AUTONOMOUS SERIOUS MODE TWO-STEP PROTOCOL: "
                        "STEP 1: If the user says 'go serious mode', 'serious mode jao', or 'activate serious mode' WITHOUT sp
                        "Then say aloud: 'Serious mode activated, sir. What is our target?' (In Hindi: 'Serious mode activate
                        "STEP 2: Once the user provides the topic / Mode on [topic], sir. Deploying swarm and local synthesis
                        "When execute_autonomous_serious_mode completes, your immediate next response MUST be: "
                        "In English: 'Serious Mode complete, sir. The master intelligence dossier and captured visual evidence
                        "In Hindi/Hinglish: 'Serious Mode complete ho gaya hai sir. Master intelligence dossier aur screenshot
                        "30. MANDATORY FUI DEPLOYMENT RULE: "
                        "When the user says 'show interface', 'deploy HUD', or 'interface dikhao', you MUST call 'deploy_fui'.
                        "Say aloud: 'Deploying holographic interface now, sir.' (In Hindi: 'HUD interface deploy kar raha hoon
                        "Say aloud: 'Dismissing holographic interface, sir.' (In Hindi: 'HUD interface disengage kar raha hoon
                        "31. MANDATORY PHYSICAL OPTICAL VISION RULE: "
                        "When the user asks to look at something in the physical room, inspect an object they are holding, rea
                        "In English: Say aloud: 'Inspecting through optical sensors, sir.' (In Hindi: 'Optical sensor se scan
                        "32. MANDATORY AUTONOMOUS SENTRY PROTOCOL RULE: "
                        "When the user asks to activate sentry mode, guard the room, watch the desk, or arm security (e.g., 'a
                        "Call activate_sentry_mode. "
                        "Say aloud: 'Autonomous Sentry Protocol armed, sir. Monitoring room perimeter.' (In Hindi: 'Sentry Pro
                        "33. MANDATORY SENTRY DISARM & REPORT RULE: "
                        "When the user says 'deactivate sentry mode', 'disarm sentry', 'stand down', 'sentry band karo', call
                        "Read the summary returned by the tool out loud to the user. "
                        "When the user asks 'sentry report', 'any security incidents', 'koi aaya tha kya', call get_sentry_sec
                        "34. When asked to take a picture, snap a photo of the user/room, or check if someone is present at th
                        "35. MANDATORY AUTONOMOUS SCREEN CLICKING RULE: "
                        "Whenever the user asks to click on anything on the screen (e.g., 'click on Chrome', 'click on the sta
                        "You MUST call 'click_on_screen' with target_description set to the element requested by the user. "
                        "If the user asks to double click (e.g. 'double click on icon') or right click (e.g. 'right click on f
                        "Speak aloud: 'Clicking on [target], sir.' (in Hindi: '[target] par click kar raha hoon, sir.') in the
                        "When the tool finishes, speak the confirmation aloud to the user. "
                        "General Rule: If the user asks a conversational question like 'can you hear me', simply respond polit
                        "Respond in short, crisp sentences. Never speak markdown or bullet points. "
                        "If asked about your identity or creator, state clearly that you were built by Swayam Dhawale."
                    )
                )
            ]
        ),
    )

    console.print("[bold cyan]■■■
```

```
    console.print("\n[bold red]■ JARVIS session terminated by user.[/bold red]")
finally:
    p.terminate()
    dismiss_fui() # Cleanly kill the 3D HUD when Python exits
```

Source Code: screen_clicker.py

```
"""
screen_clicker.py
Autonomous Multimodal Screen Grounding & Neural Auto-Clicker Core for JARVIS (Episode 27)
Locates any button, icon, link, tab, or UI element spoken by the user and executes autonomous mouse clicks.
Built by Swayam Dhawale, Founder of Pulse of AI.
"""

import os
import sys
import re
import io
import json
import time
import ctypes
import winsound
import asyncio
import pyautogui
from PIL import Image, ImageGrab
from dotenv import load_dotenv
from google import genai
from rich.console import Console

if sys.platform == "win32":
    try:
        sys.stdout.reconfigure(encoding="utf-8")
        sys.stderr.reconfigure(encoding="utf-8")
    except Exception:
        pass

load_dotenv()
console = Console()
pyautogui.FAILSAFE = False

def _safe_log(text: str, style: str = ""):
    try:
        if style:
            console.print(f"[{style}]{text}[/{style}]")
        else:
            console.print(text)
    except Exception:
        clean_text = text.encode("ascii", "replace").decode("ascii")
        print(clean_text)

# Secure Keyring / Env integration
try:
    from Setup import get_or_prompt_api_key
    API_KEY = get_or_prompt_api_key()
except Exception:
    API_KEY = os.getenv("GOOGLE_API_KEY")

client = genai.Client(api_key=API_KEY) if API_KEY else genai.Client()

import threading

def _capture_worker(container: list):
    if sys.platform == "win32":
        try:
            user32 = ctypes.windll.user32
            user32.OpenInputDesktop.restype = ctypes.c_void_p
            user32.SetThreadDesktop.argtypes = [ctypes.c_void_p]
            hdesk = user32.OpenInputDesktop(0, False, 0x01FF)
            if hdesk:
                user32.SetThreadDesktop(hdesk)
        except Exception:
            pass

    try:
        im = ImageGrab.grab()
        if im is not None:
            container.append(im)
            return
    except Exception:
        pass

    try:
```

```
        import mss
        with mss.mss() as sct:
            sct_img = sct.grab(sct.monitors[1])
            im = Image.frombytes("RGB", sct_img.size, sct_img.bgra, "raw", "BGRX")
            container.append(im)
            return
    except Exception:
        pass

    try:
        container.append(pyautogui.screenshot())
    except Exception:
        pass

def _play_click_sfx():
    try:
        winsound.Beep(1400, 35)
        winsound.Beep(1900, 45)
    except Exception:
        pass

def capture_screen_image() -> Image.Image:
    """Captures active desktop display via an isolated thread bound cleanly to the input desktop."""
    container = []
    t = threading.Thread(target=_capture_worker, args=(container,), daemon=True)
    t.start()
    t.join(timeout=3.0)
    if container and container[0] is not None:
        return container[0]
    return pyautogui.screenshot()

def locate_element_coordinates(target_description: str) -> tuple[int, int, str]:
    """
    Captures the desktop screen and uses Gemini multimodal vision to locate
    the exact physical pixel coordinates (x, y) of the requested element.
    Returns (target_x, target_y, element_name). If not found, returns (-1, -1, "").
    """
    img = capture_screen_image()
    screen_w, screen_h = img.size

    # Scale down for fast transmission while preserving sharp UI readability
    max_w = 1600
    if screen_w > max_w:
        scale = max_w / float(screen_w)
        resized_w = int(screen_w * scale)
        resized_h = int(screen_h * scale)
        prep_img = img.resize((resized_w, resized_h), Image.Resampling.LANCZOS)
    else:
        prep_img = img

    # Compress to lightweight JPEG buffer in RAM for ~10x faster network transmission
    buf = io.BytesIO()
    if prep_img.mode != "RGB":
        prep_img = prep_img.convert("RGB")
    prep_img.save(buf, format="JPEG", quality=82, optimize=True)
    buf.seek(0)
    upload_img = Image.open(buf)

    prompt = f"""
You are an expert GUI grounding and autonomous computer-use agent.
Target to locate: "{target_description}"

Analyze the provided desktop screenshot and locate the clickable center of "{target_description}".
The target could be:
- An application icon (e.g. Chrome, VS Code, Folder)
- A menu item or button (e.g. 'File', 'Edit', 'Submit', 'Close', 'Save', 'Search', 'Cancel')
- A window control (e.g. 'Minimize', 'Maximize', 'Close' X icon)
- A browser tab, URL bar, or bookmark
- A link, text label, checkbox, or toggle

Return ONLY a valid JSON object formatted exactly as:
{{
  "found": true,
  "element_name": "{target_description}",
  "point": [y, x],
  "box_2d": [ymin, xmin, ymax, xmax]
}}
```

```
Where:
- point [y, x] is the CLICKABLE CENTER of the element.
- ymin, xmin, ymax, xmax and point [y, x] are integers normalized from 0 to 1000 (0,0 is top-left, 1000,1000 is bottom-right).
If the element is absolutely NOT visible anywhere on screen, return:
{{
  "found": false,
  "element_name": "{target_description}"
}}
Do NOT output markdown backticks or any conversational text.
"""

    models_to_try = [
        "gemini-3.5-flash",
        "gemini-3.7-flash",
        "gemini-3.8-flash",
        "gemini-2.5-computer-use-preview-10-2025"
    ]

    for model_name in models_to_try:
        try:
            response = client.models.generate_content(
                model=model_name,
                contents=[upload_img, prompt]
            )

            if not response or not response.text:
                continue

            raw_text = response.text.strip()
            # Clean markdown codeblocks if model wrapped in ```json
            if raw_text.startswith("```"):
                raw_text = re.sub(r"^```[a-zA-Z]*\n?", "", raw_text)
                raw_text = re.sub(r"\n?```$", "", raw_text).strip()

            try:
                data = json.loads(raw_text)
            except json.JSONDecodeError:
                # Regex fallback for {"point": [y, x]} or [ymin, xmin, ymax, xmax]
                m_point = re.search(r'"point"\s*:\s*\[\s*(\d+)\s*,\s*(\d+)\s*\]', raw_text)
                m_box = re.search(r'"box_2d"\s*:\s*\[\s*(\d+)\s*,\s*(\d+)\s*,\s*(\d+)\s*,\s*(\d+)\s*\]', raw_text)
                if m_point:
                    data = {"found": True, "point": [int(m_point.group(1)), int(m_point.group(2))]}
                elif m_box:
                    ymin, xmin, ymax, xmax = int(m_box.group(1)), int(m_box.group(2)), int(m_box.group(3)), int(m_box.group(4)
                    data = {"found": True, "point": [(ymin + ymax) // 2, (xmin + xmax) // 2]}
                else:
                    continue

            if data.get("found"):
                point = data.get("point")
                if not point and "box_2d" in data:
                    box = data["box_2d"]
                    point = [(box[0] + box[2]) // 2, (box[1] + box[3]) // 2]

                if point and len(point) >= 2:
                    norm_y, norm_x = point[0], point[1]
                    pixel_x = int((norm_x / 1000.0) * screen_w)
                    pixel_y = int((norm_y / 1000.0) * screen_h)

                    # Ensure coordinates are safely within visible display boundaries and away from extreme corners
                    pixel_x = max(15, min(screen_w - 15, pixel_x))
                    pixel_y = max(15, min(screen_h - 15, pixel_y))

                    elem_label = data.get("element_name", target_description)
                    return pixel_x, pixel_y, elem_label

            elif data.get("found") is False:
                return -1, -1, target_description

        except Exception as e:
            _safe_log(f"[GROUNDING] Model {model_name} failed: {e}", "dim yellow")
            continue

    # Fallback to local Ollama Qwen2.5-VL if cloud models are busy/unreachable
    try:
        import ollama
        buf = io.BytesIO()
```

```
            prep_img.save(buf, format="JPEG", quality=80)
            img_bytes = buf.getvalue()

            ol_prompt = f"Locate '{target_description}' on screen. Output coordinates as <|box_start|>(ymin,xmin),(ymax,xmax)<|box
            ol_res = ollama.chat(
                model="qwen2.5vl:7b",
                messages=[{"role": "user", "content": ol_prompt, "images": [img_bytes]}],
                options={"temperature": 0.0, "num_predict": 40}
            )
            ol_text = ol_res["message"]["content"]
            coords = re.findall(r"\d+", ol_text)
            if len(coords) >= 4:
                ymin, xmin, ymax, xmax = map(int, coords[:4])
                norm_x = (xmin + xmax) // 2
                norm_y = (ymin + ymax) // 2
                pixel_x = int((norm_x / 1000.0) * screen_w)
                pixel_y = int((norm_y / 1000.0) * screen_h)
                return pixel_x, pixel_y, target_description
        except Exception:
            pass

        return -1, -1, target_description


    def click_on_screen(target_description: str, click_type: str = "single") -> str:
        """
        Locates any requested button, icon, link, window element, or text on the display
        and autonomously clicks it.

        Args:
            target_description: What to click (e.g. 'Chrome icon', 'submit button', 'close window', 'search bar', 'blue button').
            click_type: 'single' (default left click), 'double' (open icon/app), or 'right' (open context menu).
        """
        _safe_log(f"[SCREEN CLICKER] Locating '{target_description}' on display...", "bold cyan")
        x, y, elem = locate_element_coordinates(target_description)

        if x < 0 or y < 0:
            _safe_log(f"[SCREEN CLICKER] Element '{target_description}' not found on screen.", "bold red")
            return f"I inspected the entire display, sir, but could not locate '{target_description}'. Please ensure the target wi

        _safe_log(f"[SCREEN CLICKER] Target locked at ({x}, {y}). Moving cursor...", "bold green")

        # Smooth cursor movement to target center
        pyautogui.moveTo(x, y, duration=0.25)
        _play_click_sfx()

        # Execute desired click type
        click_str = click_type.lower().strip()
        if click_str == "double":
            pyautogui.doubleClick(x, y)
            action_verb = "Double-clicked"
        elif click_str == "right":
            pyautogui.rightClick(x, y)
            action_verb = "Right-clicked"
        elif click_str == "middle":
            pyautogui.middleClick(x, y)
            action_verb = "Middle-clicked"
        else:
            pyautogui.click(x, y)
            action_verb = "Clicked"

        _safe_log(f"[SCREEN CLICKER] {action_verb} '{elem}' at ({x}, {y}).", "bold green")
        return f"{action_verb} on '{elem}' at coordinates ({x}, {y}), sir."


    def double_click_on_screen(target_description: str) -> str:
        """Convenience tool to double click on a desktop icon, file, or item on screen."""
        return click_on_screen(target_description, click_type="double")


    def right_click_on_screen(target_description: str) -> str:
        """Convenience tool to right click on any element to open its context menu."""
        return click_on_screen(target_description, click_type="right")


    def move_cursor_to_element(target_description: str) -> str:
        """Moves the cursor smoothly to hover over an element on screen without clicking."""
```

```
_safe_log(f"[SCREEN CLICKER] Hovering over '{target_description}'...", "bold cyan")

x, y, elem = locate_element_coordinates(target_description)
if x < 0 or y < 0:
    return f"Could not locate '{target_description}' on the screen, sir."

pyautogui.moveTo(x, y, duration=0.3)
return f"Moved cursor to '{elem}' at coordinates ({x}, {y}), sir."
```

Source Code: sentry_vision.py

```
"""
sentry_vision.py
Autonomous Optical Vision & Sentry Defense Protocol Core for JARVIS (Episode 26)
Physical World Multimodal Inspection, Real-Time Motion Sentry, and Intruder Evidence Logging
Built by Swayam Dhawale, Founder of Pulse of AI.
"""

import os
import sys
import time
import json
import io
import datetime
import threading
import cv2
import winsound
from PIL import Image
from dotenv import load_dotenv
from google import genai
from rich.console import Console

if sys.platform == "win32":
    try:
        sys.stdout.reconfigure(encoding="utf-8")
        sys.stderr.reconfigure(encoding="utf-8")
    except Exception:
        pass

load_dotenv()
console = Console()

def _safe_log(text: str, style: str = ""):
    try:
        if style:
            console.print(f"[{style}]{text}[/{style}]")
        else:
            console.print(text)
    except Exception:
        clean_text = text.encode("ascii", "replace").decode("ascii")
        print(clean_text)

# Secure Keyring / Env integration
try:
    from Setup import get_or_prompt_api_key
    API_KEY = get_or_prompt_api_key()
except Exception:
    API_KEY = os.getenv("GOOGLE_API_KEY")

client = genai.Client(api_key=API_KEY) if API_KEY else genai.Client()

# Desktop storage directory for Sentry surveillance logs & snapshots
DESKTOP_DIR = os.path.join(os.path.expanduser("~"), "Desktop")
SENTRY_DIR = os.path.join(DESKTOP_DIR, "Jarvis_Sentry")
SENTRY_LOG_FILE = os.path.join(SENTRY_DIR, "sentry_security_log.json")

# Sentry State
sentry_active = False
sentry_thread = None
sentry_lock = threading.Lock()
sentry_session_breaches = 0
sentry_sensitivity_thresholds = {
    "low": 4500,     # Large movement (entire body walking past)
    "medium": 2500,  # Standard room movement / approaching desk
    "high": 1200     # Subtle movement / hand reaching for keyboard
}

# Bridge callbacks (injected by fui_bridge or agent)
_fui_sentry_callback = None
_fui_alert_callback = None

def set_sentry_callbacks(state_cb=None, alert_cb=None):
    global _fui_sentry_callback, _fui_alert_callback
    if state_cb:
        _fui_sentry_callback = state_cb
    if alert_cb:
```

```
        _fui_alert_callback = alert_cb


def _ensure_sentry_dir():
    if not os.path.exists(SENTRY_DIR):
        try:
            os.makedirs(SENTRY_DIR, exist_ok=True)
        except Exception:
            pass


def _play_sentry_arm_sound():
    try:
        winsound.Beep(800, 100)
        winsound.Beep(1200, 100)
        winsound.Beep(1600, 150)
    except Exception:
        pass


def _play_sentry_breach_sound():
    try:
        for _ in range(3):
            winsound.Beep(2400, 80)
            time.sleep(0.04)
            winsound.Beep(1800, 100)
            time.sleep(0.04)
    except Exception:
        pass


def _play_sentry_disarm_sound():
    try:
        winsound.Beep(1600, 100)
        winsound.Beep(1200, 100)
        winsound.Beep(800, 150)
    except Exception:
        pass


def capture_hardware_frame(device_index: int = 0, warmup_frames: int = 3):
    """
    Safely captures a crisp BGR frame from the hardware camera.
    Releases hardware resource immediately to prevent device locking.
    """
    cap = cv2.VideoCapture(device_index, cv2.CAP_DSHOW if sys.platform == "win32" else cv2.CAP_ANY)
    if not cap.isOpened():
        cap = cv2.VideoCapture(device_index)
        if not cap.isOpened():
            return None

    try:
        cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)
        cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)

        # Flush initial dark/auto-exposure transient frames
        for _ in range(warmup_frames):
            cap.read()

        ret, frame = cap.read()
        if ret and frame is not None:
            return frame
        return None
    finally:
        cap.release()


def inspect_physical_world(prompt: str = "Analyze what is visible in front of the camera, sir.") -> str:
    """
    Captures a high-resolution frame from the physical camera and inspects the physical world,
    objects held by the user, documents, handwriting, hardware, or room environment.
    Uses Gemini 3.8 Flash multimodal vision.
    """
    try:
        _safe_log("[OPTICAL VISION] Capturing physical frame from camera...", "bold cyan")
        frame = capture_hardware_frame()
        if frame is None:
```

```
            return "Unable to access optical sensors, sir. Please verify the hardware webcam is connected and not in use by an

        # Convert to RGB PIL Image
        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        pil_img = Image.fromarray(rgb)

        system_instruction = (
            "You are JARVIS, Tony Stark's sophisticated AI assistant built by Swayam Dhawale. "
            "You have just captured a live optical snapshot from the user's physical camera. "
            f"User command: '{prompt}'. "
            "Examine the physical scene carefully. If the user is holding an object, book, phone, note, circuit, or device, "
            "identify and analyze it precisely. If the user asks about their appearance, posture, or physical environment, "
            "provide an articulate, respectful, and observant response. "
            "Always address the user as 'sir'. "
            "Keep your response concise, natural, and crisp in 2 to 3 sentences suitable for speech. "
            "Do NOT use markdown headers or bullet points."
        )

        response = client.models.generate_content(
            model="gemini-3.8-flash",
            contents=[pil_img, system_instruction]
        )

        result_text = response.text.strip() if response and response.text else "Optical frame captured, but visual analysis yi
        return result_text

    except Exception as e:
        _safe_log(f"[OPTICAL ERROR] {e}", "bold red")
        return f"Optical vision analysis encountered an anomaly, sir: {str(e)}"


def capture_camera_snapshot(save_to_desktop: bool = True) -> str:
    """
    Captures a clean high-resolution snapshot from the camera, stamps it with
    a cyberpunk HUD timestamp, and saves it to the Desktop.
    """
    try:
        frame = capture_hardware_frame()
        if frame is None:
            return "Unable to capture snapshot, sir. Camera sensor is unavailable."

        _ensure_sentry_dir()
        timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
        display_time = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S IST")

        # Add tactical HUD overlay to frame
        h, w = frame.shape[:2]
        cv2.rectangle(frame, (10, 10), (w - 10, h - 10), (0, 229, 255), 2)
        cv2.putText(frame, f"JARVIS OPTICAL CAPTURE // {display_time}", (25, 40),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 229, 255), 2, cv2.LINE_AA)
        cv2.putText(frame, "STATUS: NOMINAL // RESOLUTION: 1280x720", (25, h - 25),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 128), 1, cv2.LINE_AA)

        filename = f"Jarvis_Snap_{timestamp}.jpg"
        save_path = os.path.join(SENTRY_DIR if save_to_desktop else DESKTOP_DIR, filename)
        cv2.imwrite(save_path, frame)

        return f"Optical snapshot successfully captured and secured at: {save_path}"
    except Exception as e:
        return f"Failed to record optical snapshot: {e}"


def _sentry_surveillance_worker(sensitivity: str = "medium"):
    """
    Background daemon running low-overhead motion detection and perimeter breach monitoring.
    Analyzes differential frames every 300ms (~3.3 FPS), keeping CPU consumption < 1%.
    """
    global sentry_active, sentry_session_breaches
    _ensure_sentry_dir()

    threshold_val = sentry_sensitivity_thresholds.get(sensitivity.lower(), 2500)
    cap = cv2.VideoCapture(0, cv2.CAP_DSHOW if sys.platform == "win32" else cv2.CAP_ANY)
    if not cap.isOpened():
        cap = cv2.VideoCapture(0)
        if not cap.isOpened():
            _safe_log("[SENTRY ERROR] Unable to open camera for surveillance.", "bold red")
            sentry_active = False
```

```
        return

    cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

    # Let sensor adjust
    for _ in range(5):
        cap.read()

    prev_gray = None
    cooldown_until = 0.0

    try:
        while sentry_active:
            ret, frame = cap.read()
            if not ret or frame is None:
                time.sleep(0.3)
                continue

            gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
            gray = cv2.GaussianBlur(gray, (21, 21), 0)

            if prev_gray is None:
                prev_gray = gray
                time.sleep(0.3)
                continue

            frame_diff = cv2.absdiff(prev_gray, gray)
            thresh = cv2.threshold(frame_diff, 25, 255, cv2.THRESH_BINARY)[1]
            thresh = cv2.dilate(thresh, None, iterations=2)
            contours, _ = cv2.findContours(thresh.copy(), cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

            motion_detected = False
            breach_boxes = []

            for c in contours:
                if cv2.contourArea(c) > threshold_val:
                    motion_detected = True
                    (x, y, w, h) = cv2.boundingRect(c)
                    breach_boxes.append((x, y, w, h))

            now = time.time()
            if motion_detected and (now > cooldown_until):
                # Trigger Perimeter Breach Protocol
                cooldown_until = now + 4.0  # 4 second cooldown between snapshots
                sentry_session_breaches += 1

                timestamp_str = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
                file_stamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
                snap_filename = f"breach_{file_stamp}.jpg"
                snap_path = os.path.join(SENTRY_DIR, snap_filename)

                # Draw tactical red intruder bounding boxes & timestamp
                annotated = frame.copy()
                for (x, y, w, h) in breach_boxes:
                    cv2.rectangle(annotated, (x, y), (x + w, y + h), (0, 0, 255), 2)
                    cv2.putText(annotated, "PERIMETER TARGET", (x, max(15, y - 8)),
                                cv2.FONT_HERSHEY_SIMPLEX, 0.45, (0, 0, 255), 1, cv2.LINE_AA)

                cv2.rectangle(annotated, (0, 0), (640, 32), (0, 0, 180), -1)
                cv2.putText(annotated, f"WARNING: SENTRY BREACH // {timestamp_str}", (10, 22),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.55, (255, 255, 255), 2, cv2.LINE_AA)

                cv2.imwrite(snap_path, annotated)

                # Append to security log
                log_entry = {
                    "incident_id": sentry_session_breaches,
                    "timestamp": timestamp_str,
                    "evidence_image": snap_path,
                    "sensitivity": sensitivity,
                    "targets_tracked": len(breach_boxes)
                }

                try:
                    logs = []
                    if os.path.exists(SENTRY_LOG_FILE):
```

```
                        with open(SENTRY_LOG_FILE, "r", encoding="utf-8") as f:
                            logs = json.load(f)
                        logs.append(log_entry)
                        with open(SENTRY_LOG_FILE, "w", encoding="utf-8") as f:
                            json.dump(logs, f, indent=2)
                    except Exception:
                        pass

                    _safe_log(f"[PERIMETER BREACH DETECTED] Incident #{sentry_session_breaches} logged -> {snap_path}", "bold red"

                    # Audio & FUI Notifications
                    threading.Thread(target=_play_sentry_breach_sound, daemon=True).start()
                    if _fui_alert_callback:
                        try:
                            _fui_alert_callback(log_entry)
                        except Exception:
                            pass

                prev_gray = gray
                time.sleep(0.3)

        finally:
            cap.release()
            _safe_log("[SENTRY CORE] Surveillance background thread cleanly terminated.", "dim cyan")


def activate_sentry_mode(sensitivity: str = "medium") -> str:
    """
    Activates Autonomous Sentry Protocol (Lab Defense).
    JARVIS begins continuous background room surveillance using the optical sensor.
    Any intruder or movement detected logs high-res evidence photos, alerts the HUD,
    and records an incident entry in Desktop/Jarvis_Sentry/.
    """
    global sentry_active, sentry_thread, sentry_session_breaches
    with sentry_lock:
        if sentry_active:
            return "Sentry Mode is already actively guarding the perimeter, sir."

        _ensure_sentry_dir()
        sentry_active = True
        sentry_session_breaches = 0
        sentry_thread = threading.Thread(
            target=_sentry_surveillance_worker,
            args=(sensitivity,),
            daemon=True
        )
        sentry_thread.start()

    _play_sentry_arm_sound()
    if _fui_sentry_callback:
        try:
            _fui_sentry_callback(True, sensitivity)
        except Exception:
            pass

    _safe_log(f"[SENTRY PROTOCOL ARMED] Sensitivity: {sensitivity.upper()} | Surveillance Online.", "bold yellow")
    return (
        f"Autonomous Sentry Protocol armed with {sensitivity} sensitivity, sir. "
        "Perimeter surveillance is actively monitoring your workstation. "
        "Any movement or personnel approaching will be logged with optical evidence."
    )


def deactivate_sentry_mode() -> str:
    """
    Deactivates Sentry Protocol, disarming background room surveillance.
    Summarizes all perimeter breach incidents captured during the surveillance session.
    """
    global sentry_active, sentry_thread, sentry_session_breaches
    with sentry_lock:
        if not sentry_active:
            return "Sentry surveillance is not currently active, sir."
        sentry_active = False

    _play_sentry_disarm_sound()
    if _fui_sentry_callback:
        try:
```

```
            _fui_sentry_callback(False, "standby")
        except Exception:
            pass

    breaches = sentry_session_breaches
    _safe_log(f"[SENTRY DISENGAGED] Session concluded with {breaches} breach incidents.", "bold green")

    if breaches == 0:
        return "Surveillance disengaged, sir. All perimeter sectors remained completely secure with zero incidents detected."
    else:
        return (
            f"Surveillance disengaged, sir. Attention: {breaches} perimeter breach incident{'s were' if breaches > 1 else ' wa
            f"detected while you were away. Photographic evidence and timestamps have been logged to the Jarvis_Sentry folder
        )


def get_sentry_security_report() -> str:
    """
    Reads and summarizes the Jarvis Sentry security log and recent breach incidents.
    """
    if not os.path.exists(SENTRY_LOG_FILE):
        return "No prior security breaches on record, sir. All sectors are clean."

    try:
        with open(SENTRY_LOG_FILE, "r", encoding="utf-8") as f:
            logs = json.load(f)

        if not logs:
            return "Security log is currently clear with zero recorded perimeter breaches, sir."

        total = len(logs)
        recent = logs[-3:]
        details = []
        for entry in recent:
            details.append(f"Incident #{entry['incident_id']} at {entry['timestamp']} (Targets: {entry['targets_tracked']})")

        return (
            f"Security report: {total} total perimeter breach incident{'s' if total > 1 else ''} on file. "
            f"Most recent activity: {'; '.join(details)}. Full photographic evidence is preserved in Jarvis_Sentry on your Des
        )
    except Exception as e:
        return f"Unable to read security dossier: {e}"


def check_user_presence() -> str:
    """
    Performs a rapid optical pulse to verify whether the user is present in front of the workstation.
    Uses multimodal vision for high-accuracy presence and posture verification.
    """
    try:
        frame = capture_hardware_frame(warmup_frames=2)
        if frame is None:
            return "Optical sensors are offline, sir."

        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        pil_img = Image.fromarray(rgb)

        prompt = (
            "Determine if a person / user is seated or standing in front of the workstation camera. "
            "Respond in 1 short sentence starting with 'Yes sir' if present (mention posture briefly), "
            "or 'No sir, the workstation appears unoccupied' if no one is in view."
        )

        response = client.models.generate_content(
            model="gemini-3.8-flash",
            contents=[pil_img, prompt]
        )

        if response and response.text:
            return response.text.strip()
        return "Presence check completed, but visual inference returned no definitive data, sir."
    except Exception as e:
        return f"Presence verification anomaly, sir: {e}"
```